

Organization and Prioritization of Tasks in DevOps Planning

Raphael Carvalho de Figueiredo Rocha

¹Curso de Sistemas de Informação
Universidade Federal de Rondonópolis (UFR)
Rondonópolis, MT, Brasil

raphaelrochac@gmail.com

Abstract. *This paper discusses the organization and prioritization of tasks in DevOps planning. The study highlights that DevOps efficiency depends not only on automation, but also on clear requirements, collaborative practices, supporting tools, and continuous feedback. The objective is to relate DevOps practices, tools, and pipeline design to the planning process.*

Resumo. *Este trabalho discute a organização e a priorização de tarefas no planejamento em DevOps. O estudo destaca que a eficiência em DevOps não depende apenas de automação, mas também de requisitos claros, práticas colaborativas, ferramentas de apoio e feedback contínuo. O objetivo é relacionar práticas, ferramentas e pipeline DevOps ao processo de planejamento.*

1. Introduction

DevOps é frequentemente associado à automação, à integração contínua, à entrega contínua e à aceleração do ciclo de deploy. No entanto, esses elementos técnicos não explicam sozinhos o bom funcionamento de um ambiente DevOps. Para que o fluxo seja eficiente, também é necessário que o trabalho seja bem organizado desde o início, que as tarefas sejam priorizadas de forma clara e que exista alinhamento entre os envolvidos no processo.

Neste contexto, este artigo tem como foco a organização e a priorização de tarefas no planejamento em DevOps. O tema é relevante porque ambientes DevOps operam com ciclos curtos, mudanças frequentes e necessidade constante de adaptação. Quando o planejamento é fraco, podem surgir atrasos, retrabalho, falhas de comunicação e perda de produtividade. Por outro lado, quando requisitos, prioridades e tarefas são bem estruturados, as práticas DevOps tendem a ser executadas com maior continuidade e previsibilidade.

A literatura mostra que aspectos como gerenciamento de requisitos, colaboração entre equipes, automação, monitoramento e uso de ferramentas de apoio influenciam diretamente a adoção e a eficiência de DevOps [Hernandez et al. 2023, Luz et al. 2018, Senapathi et al. 2019]. Além disso, estudos sobre planejamento de sprint e adoção organizacional de DevOps reforçam que a tomada de decisão, a estimativa de esforço e a melhoria dos processos são elementos importantes para sustentar o fluxo de trabalho [Angara et al. 2020, Diaz et al. 2021].

O objetivo deste artigo é analisar, com base em cinco estudos selecionados, como a organização e a priorização de tarefas se relacionam com práticas, ferramentas e pipeline DevOps. A proposta é mostrar que esses elementos não são apenas apoio gerencial, mas parte importante da continuidade e da eficiência do fluxo DevOps.

2. Literature review

A literatura analisada indica que DevOps não deve ser entendido apenas como um conjunto de ferramentas técnicas para automação, integração contínua e entrega contínua. Os estudos selecionados mostram que a eficiência em DevOps também depende de fatores organizacionais, como clareza dos requisitos, colaboração entre equipes, planejamento contínuo, rastreabilidade e uso adequado de ferramentas de apoio.

O estudo de Hernández, Moros e Nicolás trata do gerenciamento de requisitos em ambientes DevOps e destaca pontos como comunicação, rastreabilidade, requisitos não funcionais e integração de ferramentas à cadeia DevOps [Hernandez et al. 2023]. Essa contribuição é importante para este artigo porque a organização e a priorização das tarefas dependem diretamente da clareza dos requisitos e da visibilidade das dependências existentes no projeto.

O trabalho de Angara, Prasad e Sridevi contribui de forma mais direta para o tema do planejamento, pois aborda ferramentas de gerenciamento voltadas ao planejamento de sprint, estimativa de esforço, apoio à decisão e acompanhamento da maturidade da execução [Angara et al. 2020]. Esse estudo reforça que o planejamento em DevOps precisa ser contínuo e apoiado por mecanismos que ajudem a reduzir incertezas durante o fluxo de trabalho.

O estudo de caso apresentado por Senapathi, Buchan e Osman mostra como capacidades, práticas e desafios de DevOps aparecem em um contexto real de adoção [Senapathi et al. 2019]. A pesquisa evidencia a relação entre automação, monitoramento, entrega contínua e colaboração entre equipes. Para este artigo, esse ponto é relevante porque demonstra que decisões de planejamento tomadas no início do fluxo afetam a execução técnica das práticas DevOps.

Luz, Pinto e Bonifácio enfatizam que a adoção bem-sucedida de DevOps depende fortemente da construção de uma cultura colaborativa entre desenvolvimento e operações [Luz et al. 2018]. Nesse sentido, a automação aparece como um meio importante, mas não como o objetivo principal. A colaboração, a transparência e o compartilhamento de responsabilidades são elementos essenciais para que a organização e a priorização das tarefas funcionem de maneira mais eficiente.

Por fim, Díaz, López-Fernández, Perez e González-Prieto analisam motivos que levam empresas a adotar uma cultura DevOps, como excesso de tempo para liberar versões, silos organizacionais, falta de colaboração e baixa eficiência dos processos [Diaz et al. 2021]. O estudo também aponta resultados esperados, como aumento de produtividade, melhoria da qualidade e maior velocidade de entrega. Esses elementos reforçam que o planejamento, a organização e a priorização não são aspectos secundários, mas partes estruturais da transformação DevOps.

Em conjunto, os cinco estudos mostram que DevOps depende de uma base organizacional sólida. A automação e a velocidade de entrega produzem melhores resultados quando estão apoiadas em requisitos claros, comunicação eficiente, ferramentas adequadas e mecanismos contínuos de feedback.

3. DevOps practices

As práticas DevOps identificadas na literatura mostram que a eficiência do fluxo depende de uma combinação entre organização, colaboração e execução técnica. No contexto deste artigo, essas práticas são analisadas principalmente a partir da forma como contribuem para organizar tarefas, definir prioridades e manter o ciclo de trabalho mais contínuo.

Uma das práticas mais importantes é a colaboração entre desenvolvimento e operações. A literatura aponta que DevOps depende da redução de silos, do compartilhamento de responsabilidades e da comunicação constante entre equipes [Luz et al. 2018, Diaz et al. 2021]. Para o tema deste trabalho, essa prática é central porque a priorização das tarefas se torna mais eficiente quando as equipes envolvidas compreendem os mesmos objetivos e acompanham o andamento do fluxo de trabalho.

Outra prática relevante é o gerenciamento de requisitos. Em ambientes DevOps, os requisitos precisam ser compreendidos, rastreados e atualizados ao longo do processo, pois mudanças frequentes podem afetar diretamente a organização das tarefas [Hernandez et al. 2023]. Assim, requisitos claros ajudam a reduzir retrabalho, facilitam a tomada de decisão e permitem que as prioridades sejam definidas com maior segurança.

O planejamento de sprint e a estimativa de esforço também aparecem como práticas ligadas diretamente ao tema. O estudo de Angara, Prasad e Sridevi mostra que o planejamento em DevOps pode ser apoiado por mecanismos de estimativa, acompanhamento da execução e apoio à decisão [Angara et al. 2020]. Essas práticas ajudam a transformar demandas em tarefas executáveis, permitindo que a equipe organize o trabalho de acordo com esforço, urgência e valor esperado.

As práticas de integração contínua, testes contínuos, entrega contínua e deploy contínuo também são importantes para o fluxo DevOps. Elas permitem integrar mudanças com frequência, validar o que foi desenvolvido e entregar novas versões de forma mais rápida e controlada [Senapathi et al. 2019]. No entanto, essas práticas dependem de uma base de planejamento adequada, pois a automação técnica perde eficiência quando as tarefas não estão bem organizadas ou quando as prioridades não estão claras.

O monitoramento contínuo também exerce papel importante no ciclo DevOps. Por meio dele, a equipe obtém informações sobre falhas, desempenho e comportamento do sistema em uso [Senapathi et al. 2019]. Essas informações retornam ao planejamento e ajudam a revisar prioridades, corrigir problemas e reorganizar tarefas futuras. Dessa forma, o planejamento em DevOps não fica restrito ao início do processo, mas passa a ser alimentado continuamente pelo feedback produzido durante a operação.

Desse modo, as práticas analisadas indicam que DevOps não deve ser entendido apenas como automação de build, teste e deploy. A literatura mostra que práticas de colaboração, gerenciamento de requisitos, planejamento, estimativa, monitoramento e feedback são fundamentais para sustentar a organização e a priorização das tarefas ao longo do ciclo DevOps.

4. DevOps tools

As ferramentas DevOps identificadas nos estudos analisados aparecem como suporte para diferentes etapas do ciclo de trabalho. Elas não substituem o planejamento, mas ajudam

a tornar as tarefas mais visíveis, rastreáveis e integradas ao fluxo de desenvolvimento e operação.

No campo do planejamento e do gerenciamento de tarefas, ferramentas como Jira Software, Confluence e Azure DevOps aparecem associadas à organização do backlog, documentação de requisitos, acompanhamento de atividades e integração com a cadeia DevOps [Hernandez et al. 2023]. Para o tema deste artigo, essas ferramentas são importantes porque apoiam a definição de prioridades e permitem maior visibilidade sobre o andamento das tarefas.

No apoio ao planejamento de sprint e à tomada de decisão, a literatura menciona mecanismos como Planning Poker, classificador Naive Bayes e análise de sentimento [Angara et al. 2020]. Esses recursos se relacionam à estimativa de esforço, à avaliação da maturidade da execução e ao acompanhamento de sinais ligados à comunicação do projeto. Dessa forma, contribuem para reduzir incertezas e apoiar decisões durante o planejamento.

Na parte técnica do fluxo DevOps, ferramentas como GoCD, TeamCity, Octopus Deploy e GitHub são citadas em contexto de integração, entrega e deploy contínuos [Senapathi et al. 2019]. Essas ferramentas ajudam a automatizar etapas do processo, tornando o fluxo mais repetível e controlado. Mesmo assim, sua eficiência depende de uma organização prévia das tarefas, pois a automação funciona melhor quando o trabalho já está bem definido.

Também aparecem ferramentas e práticas ligadas à infraestrutura como código, como Terraform e pipelines compartilhados [Senapathi et al. 2019, Luz et al. 2018]. Esse tipo de recurso contribui para padronizar ambientes, reduzir configurações manuais e evitar atrasos causados por diferenças entre ambientes de desenvolvimento, teste e produção.

Na etapa de monitoramento e observabilidade, ferramentas como Datadog e New Relic ajudam a acompanhar desempenho, falhas e comportamento do sistema em operação [Senapathi et al. 2019]. Essas informações são relevantes porque geram feedback para a equipe, permitindo revisar prioridades e reorganizar tarefas futuras com base em dados reais.

Por fim, ferramentas de comunicação, como Slack e Hip Chat, aparecem relacionadas à colaboração entre equipes [Luz et al. 2018]. Elas ajudam a reduzir barreiras entre desenvolvimento e operações e facilitam a troca rápida de informações. No contexto deste artigo, isso reforça a ideia de que ferramentas DevOps não devem ser vistas apenas como recursos técnicos, mas também como apoio à coordenação, à comunicação e à priorização do trabalho.

5. DevOps pipeline

Com base nos estudos analisados e no foco deste artigo, foi elaborado um pipeline conceitual voltado à organização e priorização de tarefas no planejamento em DevOps. A proposta é mostrar que o ciclo DevOps não começa apenas na etapa de desenvolvimento ou automação, mas antes disso, com requisitos, definição de prioridades e organização do trabalho.

A Figura 1 apresenta o pipeline conceitual proposto. As três primeiras etapas foram colocadas antes do desenvolvimento porque o tema deste artigo está diretamente li-

Requirements and Prioritization → Sprint Planning → Task Organization
→ Development → Build → Test → Integration
→ Deployment → Monitoring → Feedback and Reprioritization

Figura 1. Conceptual DevOps pipeline focused on organization and prioritization

gado ao planejamento. A etapa de Requirements and Prioritization representa o momento em que demandas são compreendidas, requisitos são analisados e prioridades iniciais são definidas. Esse ponto se relaciona ao gerenciamento de requisitos, à rastreabilidade e à comunicação entre stakeholders [Hernandez et al. 2023].

Em seguida, a etapa de Sprint Planning transforma essas prioridades em um plano de execução. Nesse momento, a equipe avalia esforço, dependências, urgência e valor esperado das tarefas. Essa etapa se relaciona com práticas de planejamento de sprint, estimativa de esforço e apoio à decisão [Angara et al. 2020]. Depois disso, em Task Organization, as tarefas são distribuídas, acompanhadas e ajustadas para que o fluxo de trabalho se mantenha visível e coordenado.

Somente após essas etapas o pipeline entra nas fases mais técnicas, como Development, Build, Test, Integration e Deployment. Essas fases correspondem ao uso de práticas como integração contínua, testes contínuos, entrega contínua e deploy contínuo, frequentemente associadas à execução técnica de DevOps [Senapathi et al. 2019]. No entanto, a eficiência dessas práticas depende da qualidade da organização feita nas etapas anteriores.

Por fim, a etapa de Monitoring gera informações sobre desempenho, falhas e comportamento do sistema em operação. Essas informações alimentam Feedback and Reprioritization, permitindo que a equipe revise prioridades, corrija problemas e reorganize tarefas futuras. Dessa forma, o pipeline proposto reforça que o planejamento em DevOps não é uma atividade isolada no início do processo, mas um ciclo contínuo apoiado por feedback operacional [Senapathi et al. 2019, Diaz et al. 2021].

6. Discussion

A análise dos estudos selecionados mostra que a organização e a priorização de tarefas estão diretamente ligadas ao funcionamento de DevOps. Embora muitas discussões sobre DevOps enfatizem automação, integração contínua e deploy, a literatura analisada indica que esses elementos técnicos dependem de uma base organizacional bem definida.

O gerenciamento de requisitos aparece como um ponto importante porque ajuda a dar clareza ao que precisa ser desenvolvido, acompanhado e entregue. Quando os requisitos não são bem compreendidos ou rastreados, a equipe tende a enfrentar retrabalho, atrasos e dificuldade para definir prioridades [Hernandez et al. 2023]. Por isso, a organização das tarefas começa antes da execução técnica do pipeline.

A colaboração entre equipes também se mostra essencial. A adoção de DevOps busca reduzir silos entre desenvolvimento e operações, criando maior compartilhamento de responsabilidades e melhor comunicação [Luz et al. 2018, Diaz et al. 2021]. Essa colaboração influencia diretamente o planejamento, pois a priorização de tarefas depende de uma visão comum sobre objetivos, riscos e necessidades do projeto.

Outro ponto discutido é que as ferramentas DevOps não devem ser vistas isoladamente. Ferramentas de planejamento, automação, infraestrutura, monitoramento e comunicação cumprem funções diferentes, mas complementares. O valor dessas ferramentas surge quando elas apoiam um fluxo mais coordenado, no qual as tarefas são acompanhadas, as prioridades são revisadas e o feedback operacional retorna para o planejamento [Senapathi et al. 2019, Angara et al. 2020].

Dessa forma, o pipeline proposto neste artigo reforça que DevOps não se limita às etapas de desenvolvimento, build, teste e deploy. O ciclo começa com requisitos, priorização e organização das tarefas, e retorna a essas dimensões por meio do monitoramento e do feedback. Assim, a organização e a priorização não são apenas atividades administrativas, mas parte estrutural da eficiência em DevOps.

7. Conclusion

Este artigo analisou a organização e a priorização de tarefas no planejamento em DevOps com base em cinco estudos selecionados da literatura. A análise mostrou que DevOps não depende apenas de automação, integração contínua, entrega contínua e deploy. Esses elementos são importantes, mas funcionam melhor quando estão apoiados em requisitos claros, colaboração entre equipes, ferramentas adequadas e feedback contínuo.

As práticas analisadas indicam que o planejamento em DevOps precisa ser contínuo e adaptável. O gerenciamento de requisitos contribui para dar clareza ao trabalho, o planejamento de sprint e a estimativa de esforço ajudam na definição de prioridades, e a colaboração entre desenvolvimento e operações favorece maior alinhamento durante o processo. Além disso, o monitoramento permite que a equipe revise decisões e reorganize tarefas a partir de informações reais do sistema em operação.

As ferramentas identificadas também reforçam essa visão, pois apoiam diferentes partes do ciclo DevOps. Algumas contribuem para o planejamento e o acompanhamento das tarefas, enquanto outras apoiam automação, infraestrutura, monitoramento, testes e comunicação. Dessa forma, elas não devem ser vistas apenas como recursos técnicos, mas como instrumentos que ajudam a sustentar a organização do fluxo de trabalho.

O pipeline conceitual proposto neste artigo reforça que o ciclo DevOps começa antes do código, com requisitos, priorização e organização das tarefas, e retorna ao planejamento por meio do feedback e da repriorização. Assim, conclui-se que a organização e a priorização de tarefas exercem papel central na continuidade, na previsibilidade e na eficiência do fluxo DevOps.

Referências

- Angara, J., Prasad, S., and Sridevi, G. (2020). Devops project management tools for sprint planning, estimation and execution maturity. *Cybernetics and Information Technologies*, 20(2):79–92.
- Diaz, J., Lopez-Fernandez, D., Perez, J., and Gonzalez-Prieto, A. (2021). Why are many businesses instilling a devops culture into their organization? *arXiv preprint arXiv:2005.10388*.
- Hernandez, R., Moros, B., and Nicolas, J. (2023). Requirements management in devops environments: a multivocal mapping study. *Requirements Engineering*, 28:317–346.

- Luz, W. P., Pinto, G., and Bonifacio, R. (2018). Building a collaborative culture: A grounded theory of well succeeded devops adoption in practice. In *Proceedings of the ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, pages 1–11.
- Senapathi, M., Buchan, J., and Osman, H. (2019). Devops capabilities, practices, and challenges: Insights from a case study. *arXiv preprint arXiv:1907.10201*.